

White-label CSS Değişken Mimarisi

Bu doküman, projede kullanılacak **CSS değişken (design token) yapısını** tanımlar. Amaç; white-label destekli, esnek ama kontrolsüz özelleştirmeye izin vermeyen bir sistem kurmaktır.

Amaç

- Organizasyonlar renkleri ve temayı özelleştirebilmeli
- Tek bir renk değişikliği tüm sistemi bozmasın
- Aynı renkte görünen ama anlamı farklı olan alanlar birbirinden bağımsız kalabilsin
- Yeni gelen ekip üyeleri sistemi hızlıca anlayabilsin

Bu yüzden değişkenler **3 katmanlı** bir yapı ile ele alınır.

1. Fiziksel Renk Paleti (Ham Renkler)

Bu katman yalnızca **gerçek renk değerlerini** içerir. Anlam taşımaz.

```
:root {  
  --color-1: #0f172a;  
  --color-2: #1e293b;  
  --color-3: #2563eb;  
  --color-4: #22c55e;  
  --color-5: #eab308;  
  --color-6: #ef4444;  
  --color-7: #ffffff;  
  --color-8: #f1f5f9;  
}
```

Kurallar - Component içinde **doğrudan kullanılmaz** - Sadece anlam bazlı değişkenlere kaynak olur - White-label'da ilk değiştirilen yer burasıdır

2. Anlam Bazlı Değişkenler

Bu katman, renklerin **ne için kullanıldığını** tanımlar.

2.1 Role (Anlamsal Renkler)

Aksiyon veya mesaj anlamı taşır.

```
:root {  
  --role-primary: var(--color-3);  
  --role-success: var(--color-4);  
}
```

```
--role-warning: var(--color-5);
--role-danger: var(--color-6);
}
```

Örnek anlamlar: - primary → ana aksiyon / odak - success → olumlu sonuç - danger → hata / yanlış / risk

2.2 Surface (Yüzeyler)

Arka plan ve layout alanları için kullanılır.

```
:root {
  --surface-page-bg: var(--color-8);
  --surface-card-bg: var(--color-7);
  --surface-modal-bg: var(--color-7);
  --surface-header-bg: var(--color-1);
}
```

Not: Surface renkleri role değildir, aksiyon anlatmaz.

2.3 State (Durumlar)

State renkleri **sabit değildir**, role üzerinden türetilir.

```
:root {
  --state-hover-mix: 10%;
  --state-active-mix: 20%;
  --state-disabled-opacity: 0.5;
}
```

Kullanım:

```
background-color: color-mix(
  in srgb,
  var(--role-primary),
  black var(--state-hover-mix)
);
```

Avantaj: - Role rengi değişirse tüm state'ler otomatik uyum sağlar

3. Component Bazlı Değişkenler

Bazı componentler **bilerek global sistemden koparılır**.

Örnek: Quiz Şıkları

```
:root {
  --question-a-bg: #e53935;
  --question-b-bg: #1e88e5;
  --question-c-bg: #43a047;
  --question-d-bg: #fbc02d;
}
```

Ne zaman kullanılır? - Tek seferlik veya çok özel görsel gereksinimler varsa - Global role değişiminden etkilenmemesi isteniyorsa

4. Component → Anlam Bağlantısı

Componentler **doğrudan renk paletine bağlanmaz**.

Doğru kullanım

```
.button-primary {
  background-color: var(--btn-primary-bg);
}
```

```
:root {
  --btn-primary-bg: var(--role-primary);
}
```

Gerekirse override

```
:root {
  --btn-primary-bg: #9333ea;
}
```

Bu sayede: - Varsayılan sistem korunur - Ama component bazlı özelleştirme mümkündür

5. Tasarım Kuralları (Özet)

- Aynı görünen renkler **aynı değişken olmak zorunda değil**
- Anlamları farklıysa mutlaka **ayrı değişken** kullanılır
- Öncelik sırası:
 - Palet
 - Role / Surface
 - Component override

Mental Model (Tek Cümle)

Renkler önce palet olarak tanımlanır, anlam kazanır, componentlere bağlanır; özel durumlar component seviyesinde koparılır.

Bu Yapının Kazandırdıkları

- White-label için güvenli özelleştirme
 - Tutarlı ama esnek tasarım sistemi
 - Yeni gelen ekip üyeleri için net kurallar
-

Bu doküman proje büyüdükçe genişletilebilir (ör. typography, spacing, radius token'ları).